



Boletín 03 Intermedio — Ejercicios Prácticos

Ejercicios avanzados de matrices 2D, algoritmos de ordenación y búsqueda, manipulación con `StringBuilder`, recursividad y expresiones regulares en Java.

Bienvenido al boletín de nivel intermedio de la **Unidad 3: Excepciones, bucles, arrays y métodos**.

Este bloque de 12 ejercicios está orientado a afianzar el pensamiento algorítmico, el trabajo con matrices multidimensionales (incluyendo matrices dentadas), la optimización de texto con `StringBuilder`, el diseño de métodos recursivos y la aplicación de expresiones regulares para validación y saneamiento de datos.

Enfoque Profesional

Modulariza tu código: descompón cada ejercicio en métodos estáticos bien nombrados, valida las precondiciones con excepciones defensivas y apóyate en el depurador de IntelliJ IDEA para inspeccionar matrices y pilas recursivas.

Ejercicio 1: Matriz Transpuesta

Enunciado:

Escribe un método `public static int[][] transponer(int[][] matriz)` que reciba una matriz rectangular de dimensiones $F \times C$ y devuelva una nueva matriz transpuesta de dimensiones $C \times F$, donde la fila i pasa a ser la columna i .

Precondición: Verifica que la matriz no sea nula ni vacía y que no sea dentada.

Ejercicio 2: Suma por Filas y Columnas en Tablero Bidimensional

Enunciado:

Dada una matriz de enteros de 4 filas y 5 columnas generada aleatoriamente con valores entre 10 y 99:

- Muestra la matriz en forma de tabla perfectamente alineada.
- Muestra al final de cada fila la suma de sus elementos.
- Muestra al pie de cada columna la suma de sus elementos.
- En la esquina inferior derecha muestra la suma total de toda la matriz.

Ejercicio 3: Verificador de Matriz Simétrica

Enunciado:

Una matriz cuadrada es simétrica si es idéntica a su matriz transpuesta (es decir, `matriz[i][j] == matriz[j][i]` para todo i, j). Implementa un método `public static boolean esSimetrica(int[][] matriz)` que realice esta comprobación en el menor número de comparaciones posible (recorriendo únicamente los elementos por encima de la diagonal principal).

Ejercicio 4: Algoritmo de Ordenación por Selección (*Selection Sort*)

Enunciado:

Implementa el algoritmo clásico de selección: en cada pasada `i` del array, localiza el índice del elemento mínimo en el subarray restante `[i, n - 1]` y realiza un intercambio (*swap*) con la posición `i`. Muestra el estado del array tras cada pasada para evidenciar el proceso.

Ejercicio 5: Búsqueda Binaria Recursiva

Enunciado:

Implementa la búsqueda binaria de forma recursiva con la firma:

```
public static int busquedaBinariaRecursiva(int[] arr, int objetivo, int inicio, int fin) .
```

Devuelve el índice si se encuentra el elemento, o `-1` si no existe. Explica con comentarios cuál es el caso base y cuál es el caso recursivo.

Ejercicio 6: Cifrado César con `StringBuilder`

Enunciado:

El cifrado César desplaza cada letra del alfabeto un número fijo de posiciones `K`. Diseña un método `public static String cifrarCesar(String texto, int desplazamiento)` que:

- Utilice `StringBuilder` para la construcción eficiente de la cadena resultante.
- Conserve mayúsculas y minúsculas respetando el ciclo del abecedario (`'Z'` pasa a `'A'`).
- Deje inalterados los espacios, números y caracteres de puntuación.

Ejercicio 7: Detector de Palíndromos Robusto

Enunciado:

Una frase es un palíndromo si se lee igual de izquierda a derecha que de derecha a izquierda, ignorando mayúsculas, tildes, signos de puntuación y espacios (por ejemplo: *"Dábale arroz a la zorra el abad"*).

Escribe un método `public static boolean esPalindromo(String frase)` que limpie la cadena utilizando expresiones regulares y compruebe la simetría mediante la técnica de dos punteros.

Ejercicio 8: Fusión Ordenada de Arrays (*Merge*)

Enunciado:

Implementa un método `public static int[] fusionar(int[] a, int[] b)` que reciba dos arrays previamente ordenados en orden ascendente (de tamaños `N` y `M`) y devuelva un nuevo array ordenado de tamaño `N + M`.

Requisito crítico: El algoritmo debe resolverse en tiempo lineal `O(N + M)` recorriendo ambos arrays simultáneamente con dos punteros, sin recurrir a `Arrays.sort()`.

Ejercicio 9: Validador Completo de DNI Español

Enunciado:

Diseña un método `public static boolean validarDniCompleto(String dni)` que:

1. Verifique con Regex que el formato conste de 8 dígitos seguidos de una letra.
2. Calcule matemáticamente la letra oficial de control correspondiente a los 8 números mediante la fórmula

```
letra = "TRWAGMYFPDXBNJZSQVHLCKE".charAt(numero % 23) .
```

3. Compare la letra calculada con la letra facilitada por el usuario (sin distinguir mayúsculas de minúsculas).

Ejercicio 10: Triángulo de Pascal con Matriz Dentada

Enunciado:

El Triángulo de Pascal es una estructura numérica donde cada celda es la suma de los dos valores directamente situados sobre ella. Diseña un método `public static int[][] generarTrianguloPascal(int filas)` que devuelva un array bidimensional dentado (*jagged array*) donde cada fila `i` tenga exactamente `i + 1` columnas y los extremos valgan 1. Muestra el triángulo en consola.

Ejercicio 11: Enmascarador de Tarjetas de Crédito con Expresiones Regulares

Enunciado:

En aplicaciones de pasarelas de pago, los números de tarjeta nunca deben guardarse ni mostrarse en claro en los logs. Diseña un método `public static String enmascararTarjetas(String texto)` que detecte secuencias de 16 dígitos (escritos juntos o separados por guiones o espacios de cuatro en cuatro) y las sustituya por `****-****-****-` seguido únicamente de los últimos 4 dígitos reales.

Ejercicio 12: Conteo Recursivo de Caminos en una Cuadrícula

Enunciado:

Imagina un robot situado en la celda superior izquierda `(0, 0)` de una cuadrícula de `N × M`. El robot solo puede moverse hacia la derecha o hacia abajo. Diseña un método recursivo `public static int contarCaminos(int n, int m)` que calcule cuántos caminos posibles existen para alcanzar la esquina inferior derecha.

- ¿Cuál es el caso base cuando `n == 1` o `m == 1`?
- ¿Cuál es la llamada recursiva que divide el problema?

¿Quieres revisar el código completo?

Consulta el [Boletín 3.2 Resuelto](#) para examinar las soluciones completas con Java 25 LTS.